

Proxy-Pointer MultiModal

Structure-Aware Document Question-Answering with Visual Citations

Proof of Concept — Technical Overview

Prepared for: Senior Leadership Review

Scope: MultiModal Retrieval-Augmented Generation (RAG) pipeline

Runtime requirement: a single Google Gemini API key

Contents

- 1. Executive Summary**3
- 2. The Problem and Our Approach**3
 - 2.1 The problem with ordinary document search3
 - 2.2 Our approach: retrieve by structure, not by fragment3
- 3. How the System Works (End to End)**4
 - 3.1 Phase 1 — Ingestion (adding a document)4
 - 3.2 Phase 2 — Query (asking a question)4
- 4. How a Proxy Pointer Is Made (The Whole Process)**5
 - 4.1 What a proxy pointer contains5
 - 4.2 The step-by-step process5
- 5. Technology Used**7
- 6. Where Everything Is Stored**7
- 7. Limitations and Future Improvements**8
- 8. Glossary**8

1. Executive Summary

This document describes a working proof of concept that lets a person ask plain-English questions about their own documents and get back an answer that is grounded in those documents — including the actual charts, tables, and figures the answer relies on, shown alongside the text.

The system is a **MultiModal RAG pipeline**. “RAG” means it retrieves relevant material from your documents first, then asks a large language model to write an answer using only that material. “MultiModal” means the answer can include images, not just text.

Its distinguishing idea is **structure-aware retrieval**. Instead of cutting documents into blind text fragments, it understands each document’s outline (sections and sub-sections) and uses that outline as the unit of retrieval. Each section is identified by a breadcrumb such as **CLIP > 3 Method > 3.1 Alignment**. These breadcrumbs are called **proxy pointers**, and they are the heart of the design.

In one sentence

Structure is the anchor; the proxy pointer locates the answer; the full section plus its images is the payload delivered to the model.

Key facts for reviewers:

- Only one credential is needed to run it: a Google Gemini API key. No paid extraction service is required.
- Documents can be added by dragging PDF or Word files into the web interface; they are processed on the spot.
- Every retrieval is auditable — the extracted text, the document outline, the search index, and the proxy-pointer ledger are all stored as readable files on disk.
- The published version of this architecture was benchmarked by its authors at roughly 96% accuracy on a 20-question, 5-document technical test suite.

2. The Problem and Our Approach

2.1 The problem with ordinary document search

Most question-answering systems split a document into many small, fixed-size text chunks and store each chunk as a numerical fingerprint (an “embedding”). When a question arrives, they find the chunks whose fingerprints look most similar and feed those to the model.

The weakness is that a chunk has no memory of where it came from. A paragraph about “memory efficiency” loses the fact that it lives inside Section 4.4 of a specific paper. Important context gets lost, related figures get separated from the text that explains them, and the model can stitch together fragments that were never meant to sit side by side.

2.2 Our approach: retrieve by structure, not by fragment

This system reads the document's own outline and turns every section into a node in a tree. Each node carries three things: its breadcrumb (the proxy pointer), the full text of that section, and any figures or tables that physically appear inside it.

We search over these structural pointers. Once the best sections are identified, we load each section in full — complete text and real image files — and hand that to the model. The chunk is only a pointer; the complete section and its images are the payload.

3. How the System Works (End to End)

The system runs in two phases. Phase 1 (ingestion) happens once per document, when a file is added. Phase 2 (query) happens every time a question is asked.

3.1 Phase 1 — Ingestion (adding a document)

1. **Upload.** The user drops a PDF or Word (.docx) file into the web interface.
2. **Extract to markdown.** The file is converted into structured markdown text with image placeholders. This is done entirely with local Python libraries — no AI and no external service.
3. **Build the structure tree.** The headings in that markdown are parsed into a hierarchy of sections and sub-sections, and each image is attached to the section it sits in.
4. **Record proxy pointers.** Every section's breadcrumb is written to a separate ledger file (covered in detail in Section 4).
5. **Embed and index.** Each section is converted into a numerical fingerprint by Google's embedding model and stored in a fast similarity index (FAISS).

3.2 Phase 2 — Query (asking a question)

6. **Embed the question.** The question is turned into the same kind of fingerprint.
7. **Broad recall.** The index returns up to 200 candidate sections — deliberately wide so nothing relevant is missed.
8. **De-duplicate.** Candidates are reduced to about 50 unique sections.
9. **Re-rank (anchor-aware).** Gemini reviews short previews of each candidate and picks the best 5. If the question names a specific "Figure 5" or "Table 1", sections containing that anchor are pulled in.
10. **Load the full payload.** The 5 winning sections are loaded in full — complete text plus their actual image files.
11. **Synthesize with vision.** Gemini reads the text and looks at the images, then writes a grounded answer. It marks any figure it uses with a special tag so the interface can display it.
12. **Show the answer.** The interface renders the text, the cited figures, and the list of source breadcrumbs so every claim is traceable.

4. How a Proxy Pointer Is Made (The Whole Process)

This section explains, step by step, exactly how a proxy pointer comes into existence — what it is, where the data comes from, which file does the work, and where the result is saved. A “proxy pointer” is simply the breadcrumb address of a section of a document, for example [GaLore](#) > [4 Method](#) > [4.4 Hyperparameters](#).

4.1 What a proxy pointer contains

Each proxy pointer is a small record with these fields:

Field	Meaning
created_at	Timestamp when the pointer was created.
doc_id	Which document it belongs to (the file name, cleaned up).
node_id	A unique id for the section within that document (e.g. n12).
breadcrumb	The structural address — this is the proxy pointer itself.
level	How deep the section sits in the outline (1 = top-level heading).
snippet	The first ~150 characters of the section, used later for re-ranking.
images	List of figures/tables that physically appear in this section.
text_chars	How much text the section contains.

4.2 The step-by-step process

Step 1 — Turn the file into structured markdown

File responsible: [pp/extract.py](#). Technology used: **PyMuPDF** for PDFs and **python-docx** for Word files. No AI is involved here; the step is fast and repeatable.

- **For PDFs:** the library reads each page and measures the font size of every line. The most common size is treated as normal body text; lines that are clearly larger become headings (largest → #, next → ##, and so on). Images are pulled out of the page and a placeholder like `` is inserted where they appear.
- **For Word files:** the library reads the paragraph styles directly. A paragraph styled “Heading 1” becomes #, “Heading 2” becomes ##, embedded images are extracted in order, and tables are converted to markdown tables.

Saved to: [data/extracted_papers/<doc_id>/<doc_id>.md](#) for the text, and [data/extracted_papers/<doc_id>/figures/](#) for the images.

Step 2 — Read the headings into a structure tree

File responsible: [pp/tree_builder.py](#). The markdown is scanned line by line. Every heading opens a new section node. The program keeps a running “stack” of parent headings, so when it sees a sub-heading it knows which parent it belongs under.

The breadcrumb is built by joining the parents with “>”. For example, if the document is [GaLore](#), the current section is [4 Method](#), and a sub-section [4.4 Hyperparameters](#) appears, the

breadcrumb becomes `GaLore > 4 Method > 4.4 Hyperparameters`. **This breadcrumb is the proxy pointer.**

As each section is closed, any image placeholders found inside it (such as `figures/img_3.png`) are attached to that section, and the first ~150 characters of its text are saved as the snippet.

Saved to: `data/trees/<doc_id>.tree.json` — the full outline of the document with every section, its breadcrumb, its text, and its images.

Step 3 — Decide which sections become proxy pointers

Not every heading becomes a proxy. A section is recorded as a proxy pointer only if it actually contains body text or at least one image. Empty or purely decorative headings are skipped. This is why the number of proxies created for a document is usually smaller than the number of headings it contains — it counts only the sections that hold real content.

Step 4 — Write each proxy to the ledger file

File responsible: `pp/proxies.py` (function `record_proxies`). Each qualifying section is written as one line of JSON to a single append-only ledger.

Where the proxies are saved

`data/proxies/proxy_pointers.jsonl`

One line = one proxy pointer. “Append-only” means new pointers are added to the end as documents are processed, so the file is a running history. The web interface reads this exact file to display the proxies, and it can be downloaded as-is.

When a document is re-processed, its old lines are removed first (function `remove_doc`) so there are never duplicate pointers for the same document.

Step 5 — Make the proxy searchable

File responsible: `pp/indexer.py`. Technology used: **Google gemini-embedding-001** (1536-dimensional fingerprints) and **FAISS** (a fast similarity index).

For each section, the program combines the proxy pointer (breadcrumb) with the section text and asks the Gemini embedding model to turn it into a fingerprint. These fingerprints are stored in the FAISS index, and a companion file keeps the matching breadcrumbs, snippets, text, and image lists.

Saved to: `data/index/faiss.index` (the fingerprints) and `data/index/meta.json` (the matching section details).

The life of one proxy pointer, summarized

PDF/DOCX → markdown + images (`extract.py`)

→ section node with a breadcrumb (`tree_builder.py`)

→ written to the ledger (`proxies.py` → `proxy_pointers.jsonl`)

→ embedded and indexed for search (`indexer.py` → `faiss.index`)

5. Technology Used

The table below lists each technology, what it does in this system, and which file uses it.

Technology	What it does here	Where it is used
Streamlit	The web interface: file upload, the question box, answers, and the proxy viewer.	app.py
PyMuPDF	Reads PDFs: extracts text, detects headings by font size, pulls out images.	pp/extract.py
python-docx	Reads Word files: maps heading styles, extracts images and tables.	pp/extract.py
Google gemini-embedding-001	Turns text into 1536-number fingerprints for similarity search.	pp/gemini.py, pp/indexer.py
FAISS	Stores fingerprints and finds the most similar sections quickly.	pp/indexer.py
Google gemini-3.1-flash-lite	Re-ranks candidates and writes the final answer while looking at the images.	pp/gemini.py, pp/rag_bot.py
Python (standard)	Builds the structure tree, the proxy ledger, and ties the steps together.	pp/tree_builder.py, pp/proxies.py, pp/pipeline.py

Why local extraction matters

The original published project used a paid Adobe service to read PDFs. This build replaces that with free, local Python libraries, so the only thing the system needs to run is a single Gemini key. The trade-off is that heading detection on unusual PDF layouts is less precise than a paid service — see Section 7.

6. Where Everything Is Stored

All data lives under the `data/` folder, next to the application. Nothing is hidden in a database; every artifact is a readable file, which makes the system easy to inspect and audit.

Path	Contents
data/extracted_papers/<doc>/<doc>.md	The extracted text of the document, in markdown.

Path	Contents
data/extracted_papers/<doc>/figures/	The images (figures and tables) pulled from the document.
data/trees/<doc>.tree.json	The document's full outline: sections, breadcrumbs, text, images.
data/index/faiss.index	The numerical fingerprints used for similarity search.
data/index/meta.json	The section details that match each fingerprint.
data/proxies/proxy_pointers.jsonl	The proxy-pointer ledger — one pointer per line, viewable in the UI.

7. Limitations and Future Improvements

In the spirit of an honest proof of concept, these are the known boundaries and the natural next steps.

- **Heading detection is heuristic.** Because PDFs are read by font size, unusual or multi-column layouts can produce a flatter or noisier outline. Retrieval still works; the breadcrumbs are simply less detailed.
- **Scanned PDFs need OCR.** Image-only PDFs contain no selectable text, so text recognition (OCR) would need to be added before they can be used.
- **Answer quality depends on the model.** The system grounds answers in retrieved sections and shows its sources, but a reviewer should still use the source list to verify important claims.

Natural upgrades:

- Swap the PDF reader for an AI- or layout-model-based extractor to improve outline quality. Nothing downstream would need to change, because every later step simply consumes the markdown.
- Add OCR for scanned documents.
- Add user accounts so different teams keep separate document libraries.

8. Glossary

Term	Plain-language meaning
RAG	Retrieval-Augmented Generation. Find relevant material first, then let the model answer using only that material.
Proxy pointer	The breadcrumb address of a document section, e.g. CLIP > 3 Method > 3.1. The unit this system retrieves.
Embedding / fingerprint	A list of numbers that captures the meaning of a piece of text so similar texts can be matched.
FAISS	A library that stores fingerprints and finds the closest matches very quickly.
Anchor-aware	The system notices when a question names a specific Figure or Table and makes sure those are retrieved.

Term	Plain-language meaning
Markdown	A simple text format that marks headings, images, and tables with plain symbols.

End of document.